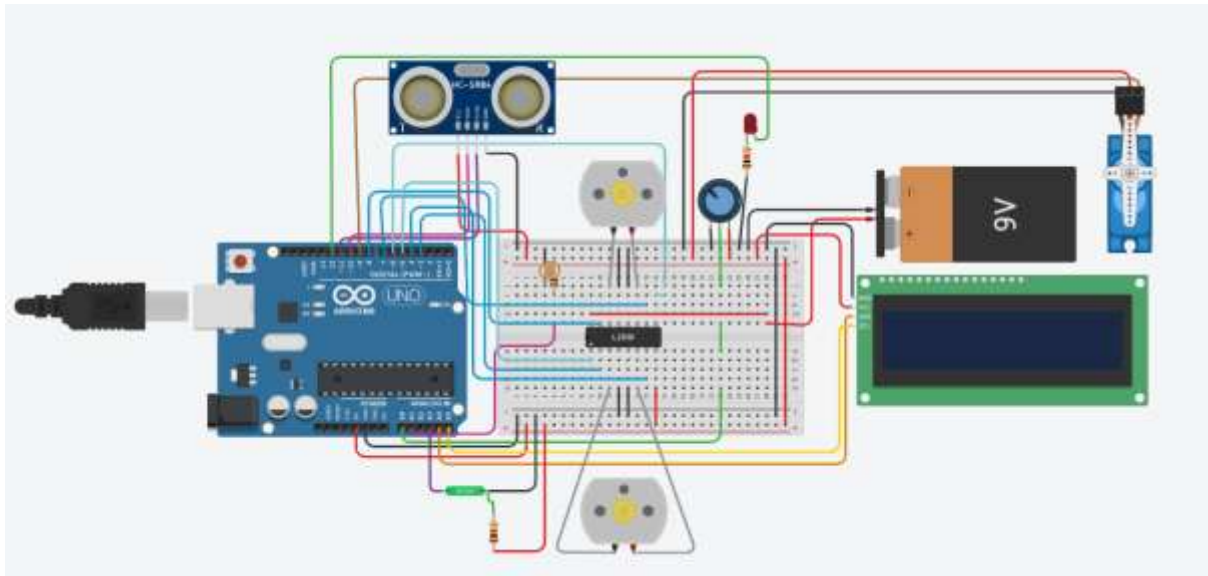




```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <Servo.h>

const byte In1 = 3;
const byte In2 = 4;
const byte Enablepin1 = 5;

const byte Enablepin2 = 6;
const byte In3 = 7;
const byte In4 = 8;

const byte Servopin = 9;

const byte Trig = 10;
const byte Echo = 11;

const byte Led = 12;

const byte Potpin = A0;
```

```
const byte Ldr = A1;
```

```
const byte Tilt = A3;
```

```
const int Minspeed = 80;
```

```
const int Maxspeed = 255;
```

```
const int Obstacleenter = 20;
```

```
const int Obstacleclear = 25;
```

```
const int Darklevel = 400;
```

```
const int Sidedifference = 10;
```

```
const unsigned long Clearforwardtime = 700;
```

```
Servo Scanservo;
```

```
LiquidCrystal_I2C Lcd(0x27, 16, 2);
```

```
enum Roverstate
```

```
{
```

```
    Moving, Scanleft, Scancenter, Scanright, Turning, Reverse, Clearforward,Safetystop};
```

```
Roverstate Statevalue = Moving;
```

```
int Speedpwm = 150;
```

```
long Frontdist = 100;
```

```
long Leftdist = 100;
```

```
long Rightdist = 100;
```

```
int Turndirection = 0;
```

```
unsigned long Statestart = 0;
```

```
unsigned long Lastdistanceread = 0;
```

```
unsigned long Lastlcd = 0;
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  pinMode(Tilt, INPUT_PULLUP);
```

```
  pinMode(In1, OUTPUT);
```

```
  pinMode(In2, OUTPUT);
```

```
  pinMode(Enablepin1, OUTPUT);
```

```
  pinMode(In3, OUTPUT);
```

```
  pinMode(In4, OUTPUT);
```

```
  pinMode(Trig, OUTPUT);
```

```
  pinMode(Echo, INPUT);
```

```
  pinMode(Led, OUTPUT);
```

```
  digitalWrite(Led, LOW);
```

```
  Scanservo.attach(Servopin);
```

```
  Scanservo.write(90);
```

```
  Lcd.init();
```

```
  Lcd.backlight();
```

```
  Lcd.clear();
```

```
  Lcd.setCursor(0, 0);
```

```
  Lcd.print("BOT READY");
```

```
  Lcd.setCursor(0, 1);
```

```
  Lcd.print("Starting");
```

```
  stopMotors();
```

```
  delay(1500);
```

```
  Lcd.clear();
```

```
  Statestart = millis();
```

```
}
```

```
void loop()
```

```
{
```

```
  readControls();
```

```
  if (digitalRead(Tilt) == LOW)
```

```
  {
```

```
    Statevalue = Safetystop;
```

```
  }
```

```
  if (Statevalue == Safetystop)
```

```
  {
```

```
    safetyStop();
```

```
    return;
```

```
  }
```

```
  updateLighting();
```

```
  switch (Statevalue)
```

```
  {
```

```
    case Moving:
```

```
      moveRobot();
```

```
      break;
```

```
    case Scanleft:
```

```
      scanLeft();
```

```
      break;
```

```
    case Scancenter:
```

```
      scanCenter();
```

```
      break;
```

case Scanright:

scanRight();

break;

case Turning:

turnRobot();

break;

case Reverse:

reverseRobot();

break;

case Clearforward:

clearForward();

break;

case Safetystop:

safetyStop();

break;

}

}

void readControls()

{

static unsigned long Lastread = 0;

if (millis() - Lastread < 50)

```
{
```

```
    return;
```

```
}
```

```
Lastread = millis();
```

```
int Potvalue = analogRead(Potpin);
```

```
Speedpwm = map(Potvalue, 0, 1023, Minspeed, Maxspeed);
```

```
Speedpwm = constrain(Speedpwm, Minspeed, Maxspeed);
```

```
}
```

```
void updateLighting()
```

```
{
```

```
    static unsigned long Lastread = 0;
```

```
    if (millis() - Lastread < 100)
```

```
    {
```

```
        return;
```

```
    }
```

```
Lastread = millis();
```

```
int Lightvalue = analogRead(Ldr);
```

```
if (Statevalue != Safetystop)
```

```
{
```

```
    if (Lightvalue < Darklevel)
```

```
{  
    digitalWrite(Led, HIGH);  
}  
else  
{  
    digitalWrite(Led, LOW);  
}  
}  
}
```

```
void moveRobot()  
{  
    if (millis() - Lastdistanceread >= 100)  
    {  
        Lastdistanceread = millis();  
  
        Frontdist = getDistance();  
  
        Serial.print("Front: ");  
        Serial.print(Frontdist);  
        Serial.println(" cm");  
    }
```

```
    if (Frontdist <= Obstacleenter)  
    {  
        stopMotors();  
        Serial.println("Obstacle detected!");  
        Serial.println("Starting scan");
```

```

    Statevalue = Scanleft;
    Statestart = millis();
    return;
}

int Adaptivespeed = Speedpwm;
if (Frontdist < 60)
{
    Adaptivespeed = map(Frontdist, Obstacleenter, 60, Minspeed, Speedpwm);
    Adaptivespeed = constrain(Adaptivespeed, Minspeed, Speedpwm);
}
setMotors(Adaptivespeed, Adaptivespeed);

showLCD("MOVING", Frontdist);
}

void scanLeft()
{
    stopMotors();
    Scanservo.write(150);
    showLCD("SCAN LEFT", 0);
    if (millis() - Statestart >= 500)
    {
        Leftdist = getDistance();

        Serial.print("LEFT: ");
        Serial.print(Leftdist);
        Serial.println(" cm");
    }
}

```

```
    Statevalue = Scancenter;
    Statestart = millis();
  }
}
```

```
void scanCenter()
{
  stopMotors();
  Scanservo.write(90);
  showLCD("SCAN CENTER", 0);
  if (millis() - Statestart >= 300)
  {
    Frontdist = getDistance();

    Serial.print("CENTER: ");
    Serial.print(Frontdist);
    Serial.println(" cm");
  }
}
```

```
    Statevalue = Scanright;
    Statestart = millis();
  }
}
```

```
void scanRight()
{
  stopMotors();
  Scanservo.write(30);
  showLCD("SCAN RIGHT", 0);
}
```

```

if (millis() - Statestart >= 500)
{
    Rightdist = getDistance();

    Serial.print("RIGHT: ");
    Serial.print(Rightdist);
    Serial.println(" cm");

    Scanservo.write(90);
    chooseDirection();
}
}

void chooseDirection()
{
    Serial.print("LEFT: ");
    Serial.print(Leftdist);

    Serial.print(" CENTER: ");
    Serial.print(Frontdist);

    Serial.print(" RIGHT: ");
    Serial.println(Rightdist);

    if (Leftdist <= Obstacleclear && Rightdist <= Obstacleclear)
    {
        Serial.println("Both sides blocked");
        Serial.println("Reversing...");
    }
}

```

```
Statevalue = Reverse;

Statestart = millis();

return;

}

if (Leftdist > Rightdist + Sidedifference)

{

    Turndirection = -1;

    Serial.println("LEFT is significantly clearer");

    Serial.println("Turning LEFT");

}

else if (Rightdist > Leftdist + Sidedifference)

{

    Turndirection = 1;


    Serial.println("RIGHT is significantly clearer");

    Serial.println("Turning RIGHT");

}


// Both sides are similar

else

{

    Serial.println("Both sides are similar");


    Turndirection = 1;


    Serial.println("Choosing RIGHT");
```

```
}
```

```
Statevalue = Turning;
```

```
Statestart = millis();
```

```
}
```

```
void turnRobot()
```

```
{
```

```
int Turnspped = constrain(Speedpwm, 100, 200
```

```
);
```

```
if (Turndirection == -1)
```

```
{
```

```
setMotors(-Turnspped, Turnspped);
```

```
showLCD("TURN LEFT", 0);
```

```
}
```

```
else
```

```
{
```

```
setMotors(Turnspped, -Turnspped);
```

```
showLCD("TURN RIGHT", 0);
```

```
}
```

```
if (millis() - Statestart >= 550)
```

```
{
```

```
stopMotors();
```

```
Scanservo.write(90);
```

```
Serial.println("Turn complete");  
Serial.println("Moving forward to clear obstacle...");
```

```
Statevalue = Clearforward;  
Statestart = millis();  
}  
}
```

```
void clearForward()  
{  
    setMotors(Speedpwm,Speedpwm);  
    showLCD("CLEARING", 0);  
  
    if (millis() - Statestart >= Clearforwardtime)  
    {  
        stopMotors();  
  
        Serial.println("Obstacle cleared");  
        Serial.println("Returning to MOVING");
```

```
        Statevalue = Moving;  
        Statestart = millis();  
        Lastdistanceread = 0;  
        Frontdist = Obstacleclear + 1;  
    }  
}
```

```
void reverseRobot()
```

```
{  
    setMotors(-Speedpwm, -Speedpwm);  
    showLCD("REVERSING", 0);  
  
    if (millis() - Statestart >= 450)  
    {  
        stopMotors();  
  
        Serial.println("Reverse complete");  
        Serial.println("Turning RIGHT");  
  
        Turndirection = 1;  
        Statevalue = Turning;  
        Statestart = millis();  
    }  
}  
long getDistance()  
{  
    long Total = 0;  
  
    for (int I = 0; I < 3; I++)  
    {  
        digitalWrite(Trig, LOW);  
  
        delayMicroseconds(2);  
  
        digitalWrite(Trig, HIGH);
```

```
delayMicroseconds(10);
```

```
digitalWrite(Trig, LOW);
```

```
long Duration = pulseIn(Echo,HIGH,25000);
```

```
long Dist;
```

```
if (Duration == 0)
```

```
    Dist = 400;
```

```
else
```

```
    Dist = Duration * 0.017;
```

```
Total += Dist;
```

```
delay(5);
```

```
}
```

```
return Total / 3;
```

```
}
```

```
void setMotors(
```

```
    int Left,
```

```
    int Right
```

```
)
```

```
{
```

```
    if (Left > 0)
```

```
    {
```

```
        digitalWrite(In1, HIGH);
```

```
        digitalWrite(In2, LOW);
```

```
    }
```

```
else if (Left < 0)
```

```
{  
    digitalWrite(In1, LOW);  
    digitalWrite(In2, HIGH);  
}
```

else

```
{  
    digitalWrite(In1, LOW);  
    digitalWrite(In2, LOW);  
}
```

if (Right > 0)

```
{  
    digitalWrite(In3, HIGH);  
    digitalWrite(In4, LOW);  
}
```

else if (Right < 0)

```
{  
    digitalWrite(In3, LOW);  
    digitalWrite(In4, HIGH);  
}
```

else

```
{  
    digitalWrite(In3, LOW);  
    digitalWrite(In4, LOW);  
}
```

```
analogWrite(Enablepin1, constrain(abs(Left), 0, 255));
```

```
analogWrite(Enablepin2, constrain(abs(Right), 0, 255));
```

```
}
```

```
void stopMotors()
```

```
{
```

```
    setMotors(0, 0);
```

```
}
```

```
void safetyStop()
```

```
{
```

```
    stopMotors();
```

```
    digitalWrite(Led, HIGH);
```

```
    showLCD("SAFETY STOP", 0);
```

```
    Serial.println("SAFETY STOP");
```

```
    if (digitalRead(Tilt) == HIGH)
```

```
    {
```

```
        digitalWrite(Led, LOW);
```

```
        Statevalue = Moving;
```

```
        Statestart = millis();
```

```
    Serial.println("Safety cleared");  
}  
}
```

```
void showLCD(  
    const char* Message,  
    long Dist  
)  
{  
    if (millis() - Lastlcd < 200)  
    {  
        return;  
    }  
}
```

```
Lastlcd = millis();  
Lcd.setCursor(0, 0);  
Lcd.print("      ");  
Lcd.setCursor(0, 0);  
Lcd.print(Message);  
Lcd.setCursor(0, 1);  
Lcd.print("      ");  
Lcd.setCursor(0, 1);
```

```
if (Dist > 0)  
{  
    Lcd.print("D:");  
    Lcd.print(Dist);  
}
```

```
Lcd.print(" S:");  
Lcd.print(Speedpwm);  
}  
  
else  
{  
    Lcd.print("Speed:");  
    Lcd.print(Speedpwm);  
}  
}
```